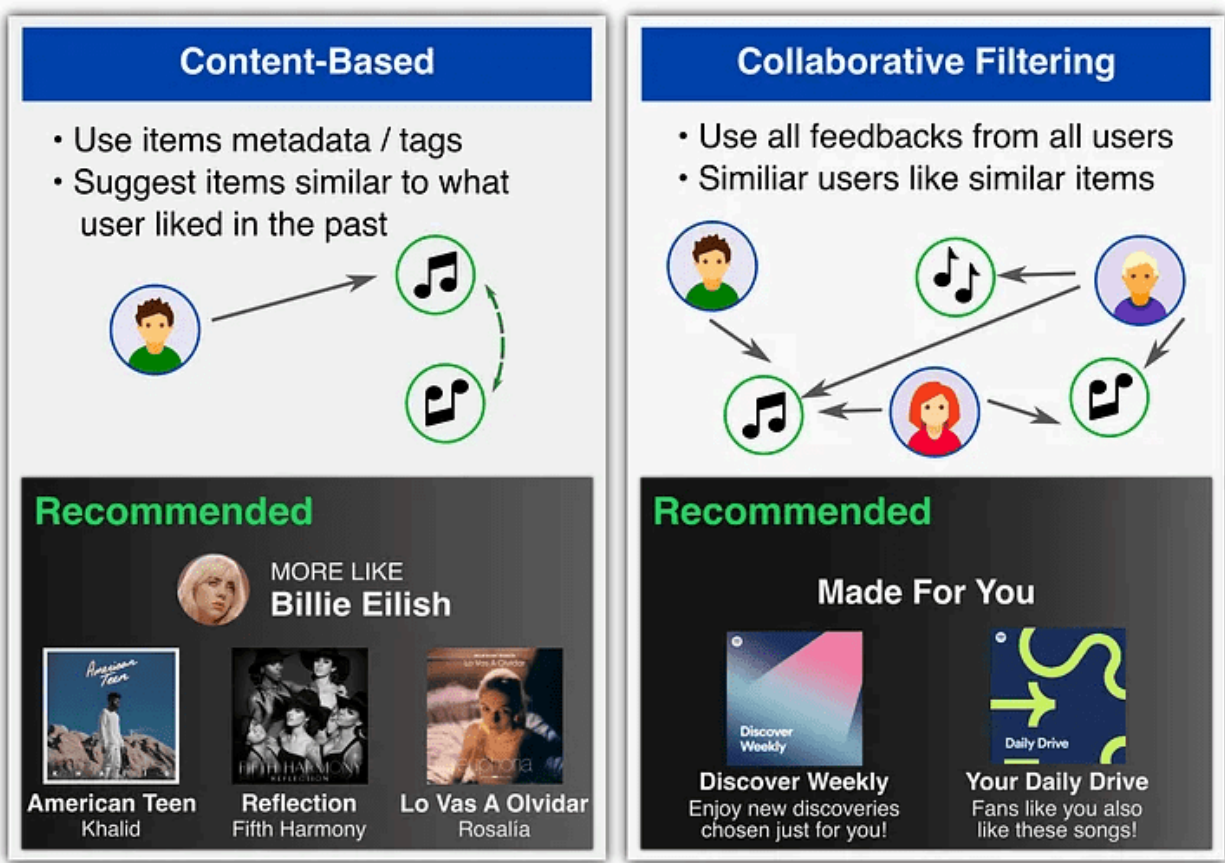


Recommendation Methods

For content based recommendation, we use two main methods:

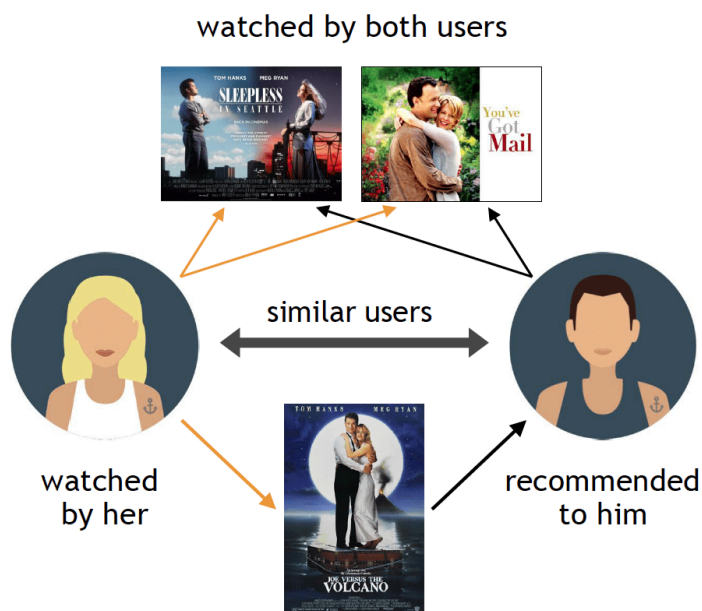
1. **Cosine Similarity** – This method calculates the cosine of the angle between the item profile vector and the user profile vector. A smaller angle (or higher cosine similarity) suggests a more robust match between the user's preferences and the item's characteristics.
2. **Classification Models** – Methods like decision trees can be used to decide whether an item would appeal to a user based on various characteristics of the user and the item.



Collaborative Filtering

Collaborative filtering (CF) is a popular technique in recommendation systems that predicts a user's preferences by using the collective knowledge and behaviors of a large pool of users. Unlike content-based filtering, which relies on the properties of items, CF focuses on user-item interactions, utilizing the feedback or activity history of all users to make predictions.

Collaborative Filtering



Memory-Based vs. Model-Based Approaches

1. Memory-Based CF – Uses the entire user-item interaction matrix to make direct recommendations based on similarities between users or items. It is straightforward but can struggle with large, sparse matrices. Generally, it

deals with implicit feedback. It consists of two methods: user-based collaborative filtering and item-based collaborative filtering.

2. Model-Based CF – Uses machine learning models to predict interactions between users and items. Techniques like matrix factorization, clustering, SVD, and deep learning are used to learn latent features from the data, improving prediction accuracy. We also use these methods for explicit feedback.

User-Based Collaborative Filtering

This method recommends items to a user based on the preferences of similar users in the database. It involves creating a matrix of items rated/liked/clicked by each user, computing similarity scores between users, and then suggesting items that the target user hasn't encountered yet but similar users have liked.

Example – If User A likes 'Batman Begins' and 'Justice League,' and User B likes 'Batman Begins,' 'Justice League,' and 'Avengers,' User A might also enjoy 'Avengers.'

Item-based Collaborative Filtering

Instead of finding similar users, this method finds similar items to recommend based on the user's past preferences. It identifies pairs of items rated/liked by the same users, measures their similarity, and then suggests similar items based on these similarity scores.

Example – If User A liked ‘Movie 1’ and ‘Movie 2’, and ‘Movie 2’ is similar to ‘Movie 3’ based on ratings by other users, ‘Movie 3’ is recommended to User A.

K-Nearest Neighbors (KNN) – To make a recommendation, KNN identifies the “neighbors” of a user or item, which are most similar based on certain features or past behavior. For example, KNN might compare users based on their movie ratings in a movie recommendation system. If a user has given similar ratings to a set of movies as another user, KNN considers them neighbors. The system then recommends movies that the neighbor liked, but the user hasn’t seen yet.

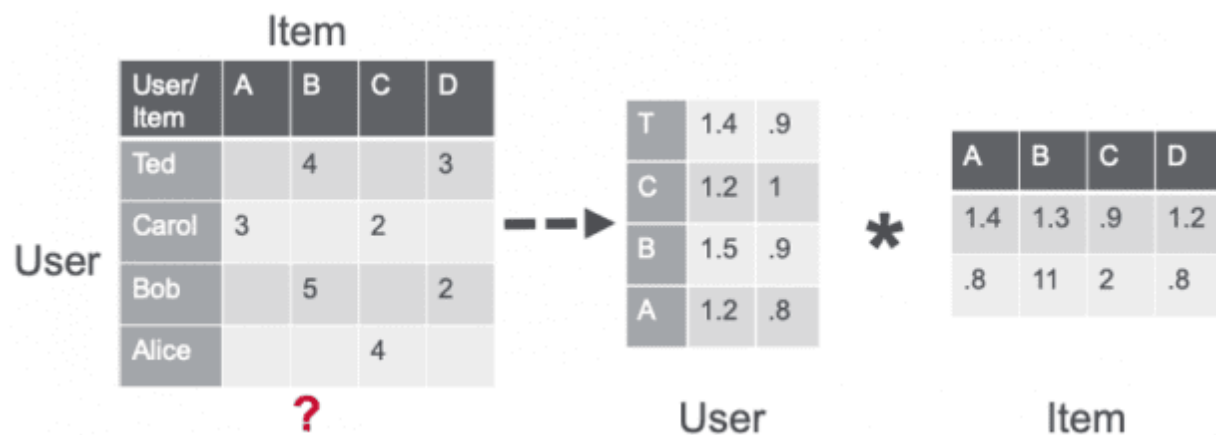
Context Filtering

Session-based and sequence-based recommender systems predict the next user action by analyzing past interactions. Session-based systems focus on the current session’s actions, while sequence-based systems use the order of all past interactions. These systems face challenges like varying session lengths, action types, and user anonymity. Conventional methods include K-nearest neighbors and Markov chains, while advanced approaches use deep learning models like RNNs and [attention mechanisms](#). Netflix, for example, uses contextual sequence prediction by considering user actions and context to suggest what to watch next.

Many advancements have occurred throughout the decades for recommendation systems. Here, we will explore three main and widely used ML techniques.

Matrix Factorization

As discussed, matrix factorization decomposes the matrix into two lower-dimensional matrices representing users and items. These lower-dimensional matrices capture latent features that describe the preferences of users and the characteristics of items. After that, latent features are used to predict user recommendations. Now, two different types of algorithms have been used based on the feedback we received.



For Implicit Feedback

The SVD (singular value decomposition) algorithm is used for implicit feedback, which involves measuring user confidence in their preferences. Suppose t_{ui} represents how much of a movie (i) user (u) has watched. For example, $t_{ui} = 0$ means the user never watched the movie, $t_{ui} = 0.3$ means they watched 30% of it, and $t_{ui} = 2$ means they watched it twice. This implies that a

user is likely more interested in a movie they've watched two times than in one they haven't watched at all.

We define the confidence matrix c_{ui} and the rating matrix r_{ui} as follows: The confidence matrix is $c_{ui} = 1 + \alpha t_{ui}$, where α is a constant. The rating matrix r_{ui} is defined as $r_{ui} = 1$ if $t_{ui} > 0$ (indicating the user has watched the movie) and $r_{ui} = 0$ if $t_{ui} = 0$ (indicating the user has not watched the movie).

$$c_{ui} = 1 + \alpha t_{ui} \quad \text{and} \quad r_{ui} = \begin{cases} 1 & \text{if } t_{ui} > 0 \\ 0 & \text{if } t_{ui} = 0 \end{cases}$$

Image 18 –

confidence matrix and rating matrix for implicit feedback

Then, using both matrices, t computes the loss of overall user-item pairs (u, i) using a weighted squared error between the observed rating r_{ui} and the predicted rating $\hat{r}_{ui}$, weighted by the confidence c_{ui} . Regularization terms for user and item feature vectors (p_u and q_i) are added to prevent overfitting and scaled by a parameter λ . If a user never interacted with an item, r_{ui} is set to 0.

$$L = \sum_{\text{all } u, i} \left[c_{ui} (r_{ui} - \hat{r}_{ui})^2 + \lambda (\|p_u\|_2^2 + \|q_i\|_2^2) \right]$$

Image 19 – Loss function for implicit feedback

For Explicit feedback

Explicit feedback uses direct user ratings for items, such as movie ratings. In matrix factorization for explicit feedback, a linear model represents user-item

interactions, and the algorithm, called probabilistic matrix factorization, learns latent vectors for users and items by minimizing a regularized mean squared error (MSE) loss over known ratings.

$$\hat{r}_{ui} = q_i^T p_u$$

$$L = \sum_{(u,i) \in K} \left[(r_{ui} - \hat{r}_{ui})^2 + \lambda \left(\|p_u\|_2^2 + \|q_i\|_2^2 \right) \right]$$

Image 20

– Probabilistic Matrix Factorization and Loss function for explicit feedback

Two common optimization methods are used: stochastic gradient descent (SGD) and alternating least squares (ALS). While SGD is easy to implement, it may struggle with non-convex loss functions. ALS, however, can transform the problem into a series of convex linear regression problems, which are easier to solve and can be significantly parallelized for speed.

[Home](#) > [Beginners](#) > [Mastering Recommendation System: A Complete Guide](#)

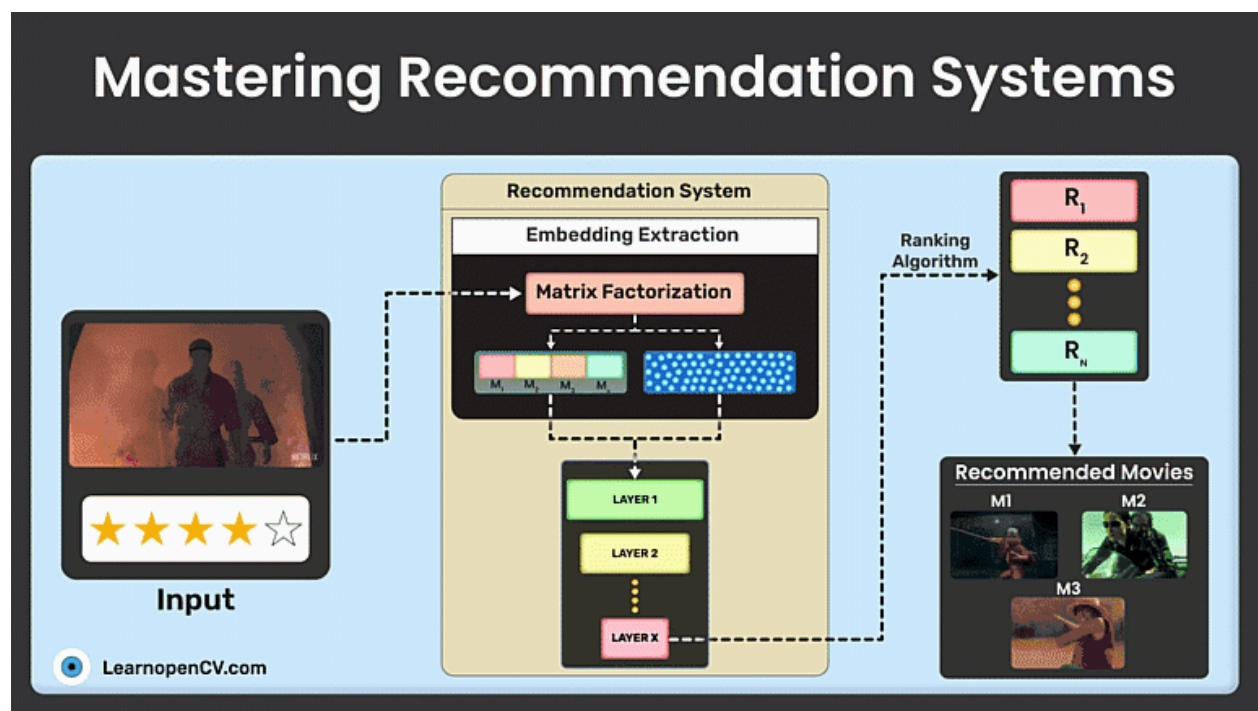
Mastering Recommendation System: A Complete Guide



Ankan Ghosh

May 21, 2024 55 Comments

Suppose you're listening to a song on Spotify, watching a video on YouTube or Netflix, or shopping on Amazon; you'll always see a list of similar songs, videos, or products recommended to you. Amazingly, most of them perfectly match your tastes. Pretty cool, right? Under the hood, these applications use a recommendation system (or recommender system) to recommend their products to the user. This article will explore what a Recommendation System is and how it has advanced over recent years. Whether you're a novice or an expert in recommendation systems, you'll thoroughly understand their workflow by the end of this blog.



To see the key takeaways, scroll down to the concluding part of the article or [click here](#) to see them immediately.

- [What is a Recommendation System?](#)
- [Types of Recommendation System](#)
- [Recommendation System – Traditional ML Approaches](#)
- [Recommendation System – Latest DL Approaches](#)
- [Key Takeaways](#)
- [Conclusion](#)
- [References](#)

What is a Recommendation System?

With the massive growth of online content, users have been overflowed with choices. Recommender systems are designed to understand users' and products' preferences, past decisions, and characteristics by analyzing interaction data such as impressions, clicks, likes, and purchases. These systems can recommend various products or services that match consumers' interests, including books, videos, products, and clothing.



Image 1- Popular Recommendation Systems

A Statistical Overview

Now, here is a list containing all the statistical data of popular companies that use recommendation systems as a main component of their business:

1. YouTube – People upload 500 hours of videos every minute, so it would take 82 years for a user to watch all the videos uploaded in the last hour.
2. Spotify – Users can listen to over 100 million song tracks and podcasts.
3. Amazon – Users can buy more than 350 million different products.
4. Netflix – If a user wants to watch all the series on Netflix, it will take approximately 10.27 years of continuous viewing, assuming no breaks and

watching 24 hours a day. It would take about 61.64 years of watching 4 hours per day to watch all the series available on Netflix.

5. Facebook – As of 2024, Facebook has approximately 3.05 billion monthly active users, and it's impossible for anyone to connect with all of them.
6. LinkedIn – Approximately 61 million people use LinkedIn to search for jobs every week. LinkedIn hosts over 15 million job listings available for users to apply to at any given time. Additionally, around 8.72 million job applications are submitted daily, translating to about 363,600 job applications per hour or 140 job applications every second.

Now, we can clearly see that every user needs suggestions or recommendations to meet their needs without wasting more time. Here, recommendation systems do the work for you and recommend all your favorite songs, the products you need to buy, or the web series you want to watch. All these platforms use powerful machine learning models to generate relevant recommendations for each user. Now, let's move to the next section to explore what popular recommendation system types we use.

TensorFlow – Keras Bootcamp

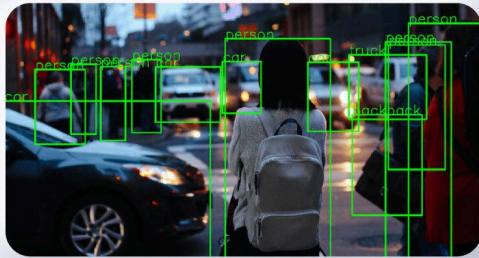


15K+ Learners

3 Hours of Learning

Join Free TensorFlow Bootcamp

PyTorch Bootcamp



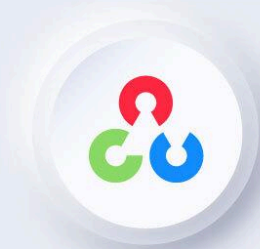
Getting Started with PyTorch

10K+ Learners

8 Hours of Learning

[Join Free PyTorch Bootcamp](#)

OpenCV Bootcamp



Learn Computer Vision & AI Using OpenCV

100K+ Learners

3 Hours of Learning

Join Free OpenCV Bootcamp

TensorFlow – Keras Bootcamp

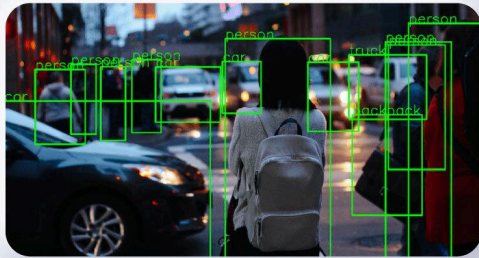


15K+ Learners

3 Hours of Learning

[Join Free TensorFlow Bootcamp](#)

PyTorch Bootcamp



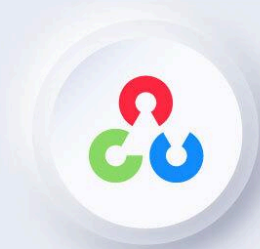
Getting Started with PyTorch

10K+ Learners

8 Hours of Learning

[Join Free PyTorch Bootcamp](#)

OpenCV Bootcamp



Learn Computer Vision & AI Using OpenCV

100K+ Learners

3 Hours of Learning

Join Free OpenCV Bootcamp

TensorFlow – Keras Bootcamp



15K+ Learners

3 Hours of Learning

Join Free TensorFlow Bootcamp

[View all AI Free Courses](#)

Types of Recommendation System

Three main types of recommendation systems are widely used:

1. Content-Based
2. Collaborative Filtering
3. Context filtering

These recommendation systems rely on feedback data regarding ratings, clicks, likes or dislikes, etc. Before we move to the types, let's explore the two main types of feedback:

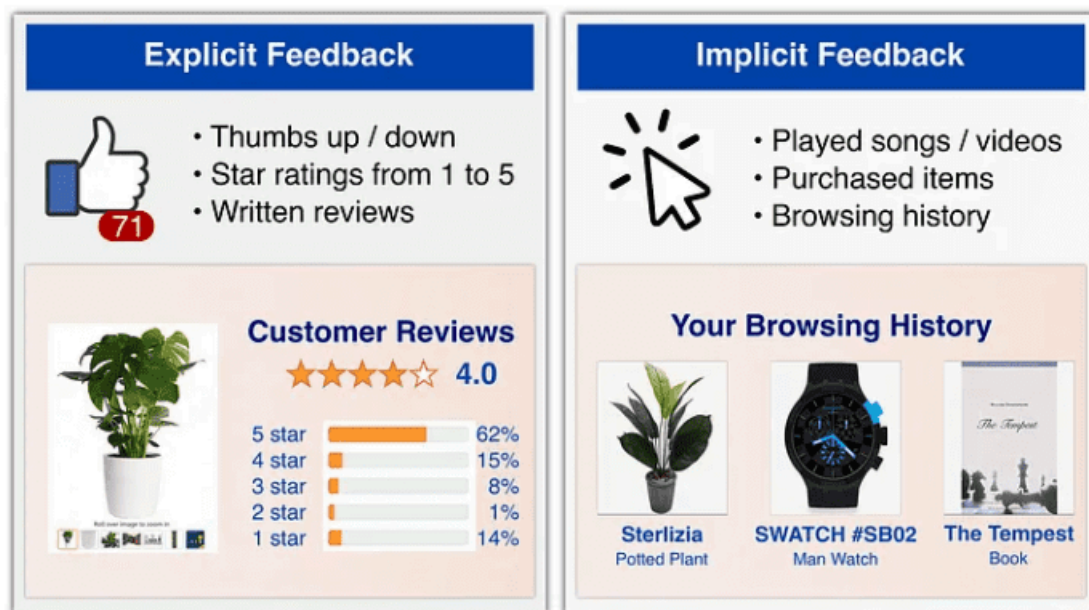


Image 2- Explicit vs Implicit Feedback

Recommendation System – Explicit Feedback

Explicit feedback is when a user directly rates something to show how much they like it. Suppose you give a product a star rating from 1 to 5 after buying it or give a video a thumbs up or down. This type of feedback gives clear information about how much a user liked something, but it is hard to collect because most people don't take the time to write reviews or rate every item they use.

Recommendation System – Implicit Feedback

Implicit feedback is based on user actions, like what they buy, browse, or listen to. Let's say a user's purchase history, the songs they play, or the videos they watch. This feedback is abundant and easy to collect, but it is less precise and can be noisy. For instance, someone might buy a product as a gift, not for themselves. Despite this noise (the model will consider the product as a preference of the user who placed the order), the large amount of data makes it very useful, and most modern recommendation systems prefer to use implicit feedback.

Rating Matrix

Rating Matrix r_{ui}					
Explicit Feedback			Implicit Feedback		
r_{ui} = star rating 1 to 5			r_{ui} = did user watch the movie?		
					
	?	?	4	?	1
	4	?	?	?	?
	?	?	?	3	2
	1	?	?	?	?
					
	1	0	1	0	1
	1	1	0	1	1
	0	1	0	1	1
	1	0	0	1	0

Image 3 – Rating Matrix

Once we collect explicit or implicit feedback, you can create a user-item rating matrix out of it (r_{ui}). For explicit feedback, each entry in (r_{ui}) is a numerical value, like the number of stars you gave to a movie or a “?” if you didn’t rate that movie. For implicit feedback, the values in (r_{ui}) are boolean, showing whether there was an interaction or not—like whether you watched a movie or not. It’s important to note that this matrix (r_{ui}) is very sparse because you only interact with a few items out of all available ones, and you rate even fewer items out of them!

Now, let's get back to the types of recommendation systems:

Content-Based Recommendation System

Content-based recommendation systems(or recommender systems) focus on the characteristics of items and users' preferences as expressed through their interactions with items. For example, in a movie recommendation system like IMDb or Netflix, each movie might be tagged with genres such as "action" or "comedy." Similarly, users are profiled based on their personal details like age and gender or their previous interactions with movies. This type of system recommends items by matching the attributes of items with users' preferences. If a user has enjoyed movies like "Batman Begins" and "The Dark Knight" the system might suggest a similar movie, "The Dark Knight Rises," because it features the same actors or belongs to a similar genre.

Content-based Filtering

watched by user



Image 4 – Content-Based Filtering

Utility Matrix

This is a structured format where each cell represents the interaction between a user and an item, such as a rating or a like. It helps understand which items a user prefers over others, although not all interactions are always known. The rating matrix we saw before is an example of this only, where we collect feedback from the user.

Creating Item Profiles

Each item in the system is described by a profile that captures its essential characteristics. For movies, this could include details about the cast, the director, the year it was released, and its genres. This information is quantified using techniques such as TF-IDF vectorization:

Term Frequency (TF): This is a count of how often a particular word appears in a document relative to the most frequent word in that document.

$$TF_{ij} = \frac{f_{ij}}{\max_k f_{kj}}$$

Image 5 – Term Frequency

Inverse Document Frequency (IDF): This calculates how unique a word is across all documents; more unique words are given higher importance.

$$IDF_i = \log_e \frac{N}{n_i}$$

Image 6 – Inverse Document Frequency

Term Frequency-Inverse Document Frequency (TF-IDF) is a number used in searching and natural language processing(NLP). It helps to find how important a word is in a document when you look at a bunch of documents together (called a corpus). TF measures how often a word appears in a document. IDF looks at how rare a word is across all documents, giving more weight to unique words. By multiplying TF and IDF, you get the TF-IDF score, which tells you how important each word is in that document.

$$TF - IDF score(w_{ij}) = TF_{ij} * IDF_i$$

Image 7 – Term

Frequency-Inverse Document Frequency

Creating User Profiles

User profiles are built from data on how users have interacted with items. These profiles are essentially vectors that summarize a user's likes and dislikes based on past behavior. The profiles help predict what new items a user might like.

Recommendation Methods

For content based recommendation, we use two main methods:

1. Cosine Similarity – This method calculates the cosine of the angle between the item profile vector and the user profile vector. A smaller angle (or higher cosine similarity) suggests a more robust match between the user's preferences and the item's characteristics.
2. Classification Models – Methods like decision trees can be used to decide whether an item would appeal to a user based on various characteristics of the user and the item.

When you search for a movie on Netflix or a photo on Google, you will get similar search results to your desired movie or photo, right? All of these recommendations are using these methods under the hood. The Recommendation System uses similarity search techniques to search and list the nearby feature vectors of your given input. We will explore this more in one of our future articles.

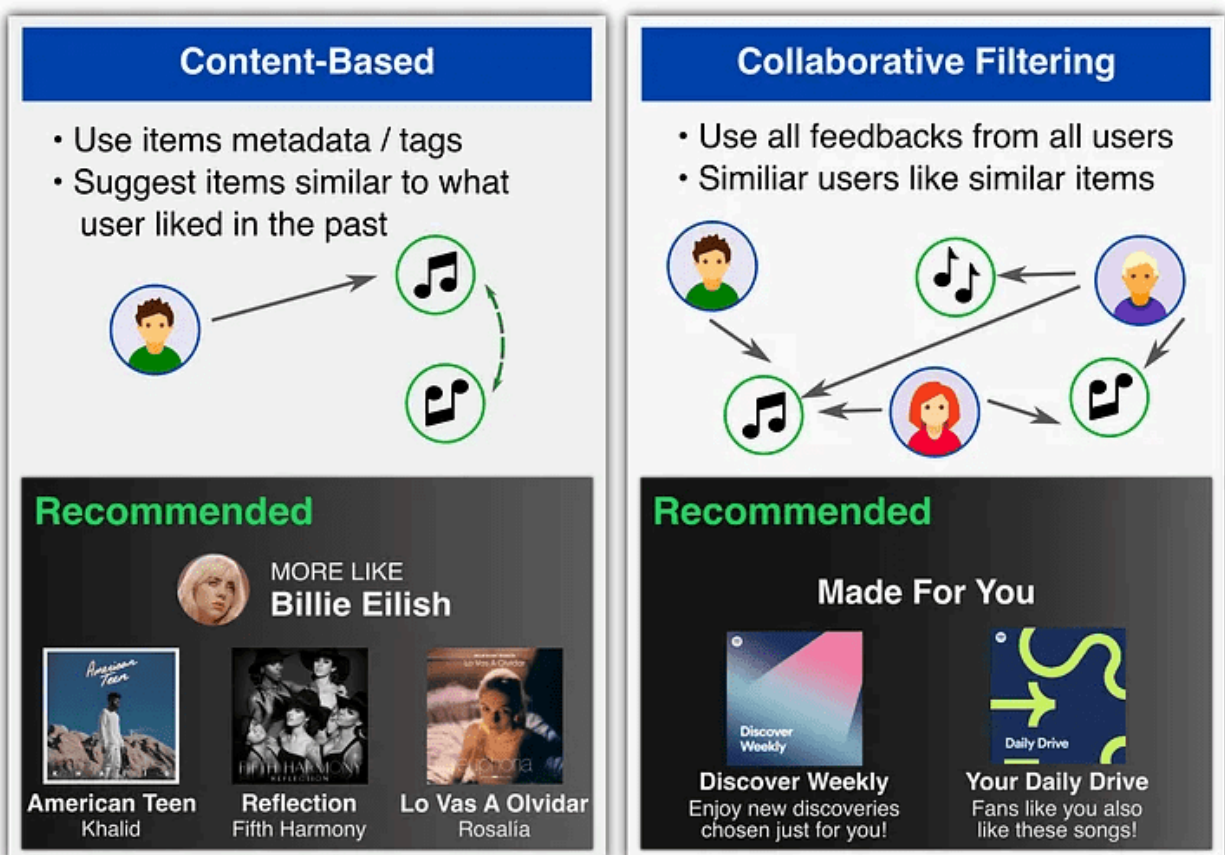


Image 8 – Content-Based vs Collaborative Filtering

Collaborative Filtering

Collaborative filtering (CF) is a popular technique in recommendation systems that predicts a user's preferences by using the collective knowledge and behaviors of a large pool of users. Unlike content-based filtering, which relies on the properties of items, CF focuses on user-item interactions, utilizing the feedback or activity history of all users to make predictions.

Collaborative Filtering

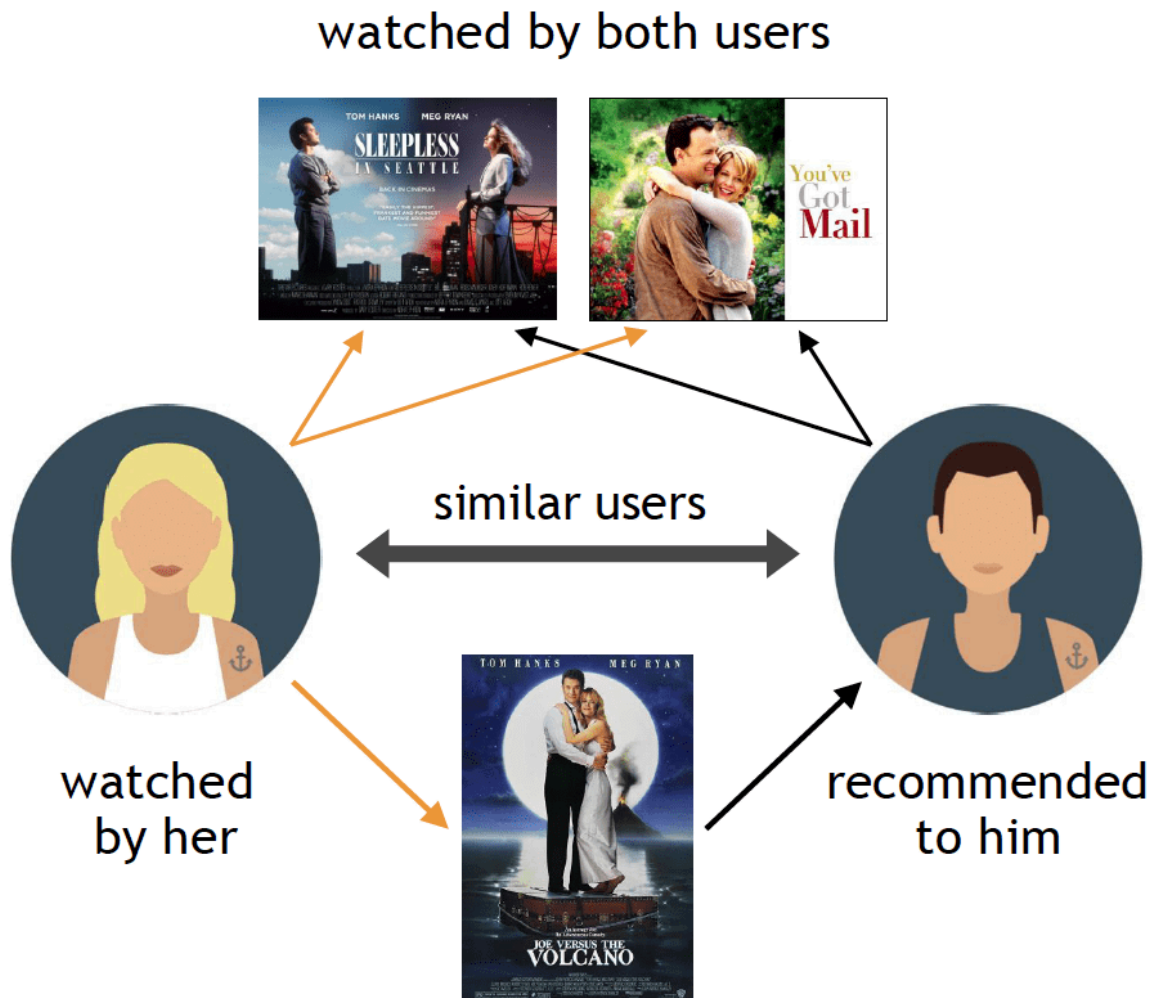


Image 9 – Collaborative Filtering

We can classify collaborative filtering (CF) recommendation systems based on various approaches. In the next section, we will explore this further.

Memory-Based vs. Model-Based Approaches

1. Memory-Based CF – Uses the entire user-item interaction matrix to make direct recommendations based on similarities between users or items. It is straightforward but can struggle with large, sparse matrices. Generally, it deals with implicit feedback. It consists of two methods: user-based collaborative filtering and item-based collaborative filtering.
2. Model-Based CF – Uses machine learning models to predict interactions between users and items. Techniques like matrix factorization, clustering, SVD, and deep learning are used to learn latent features from the data, improving prediction accuracy. We also use these methods for explicit feedback.

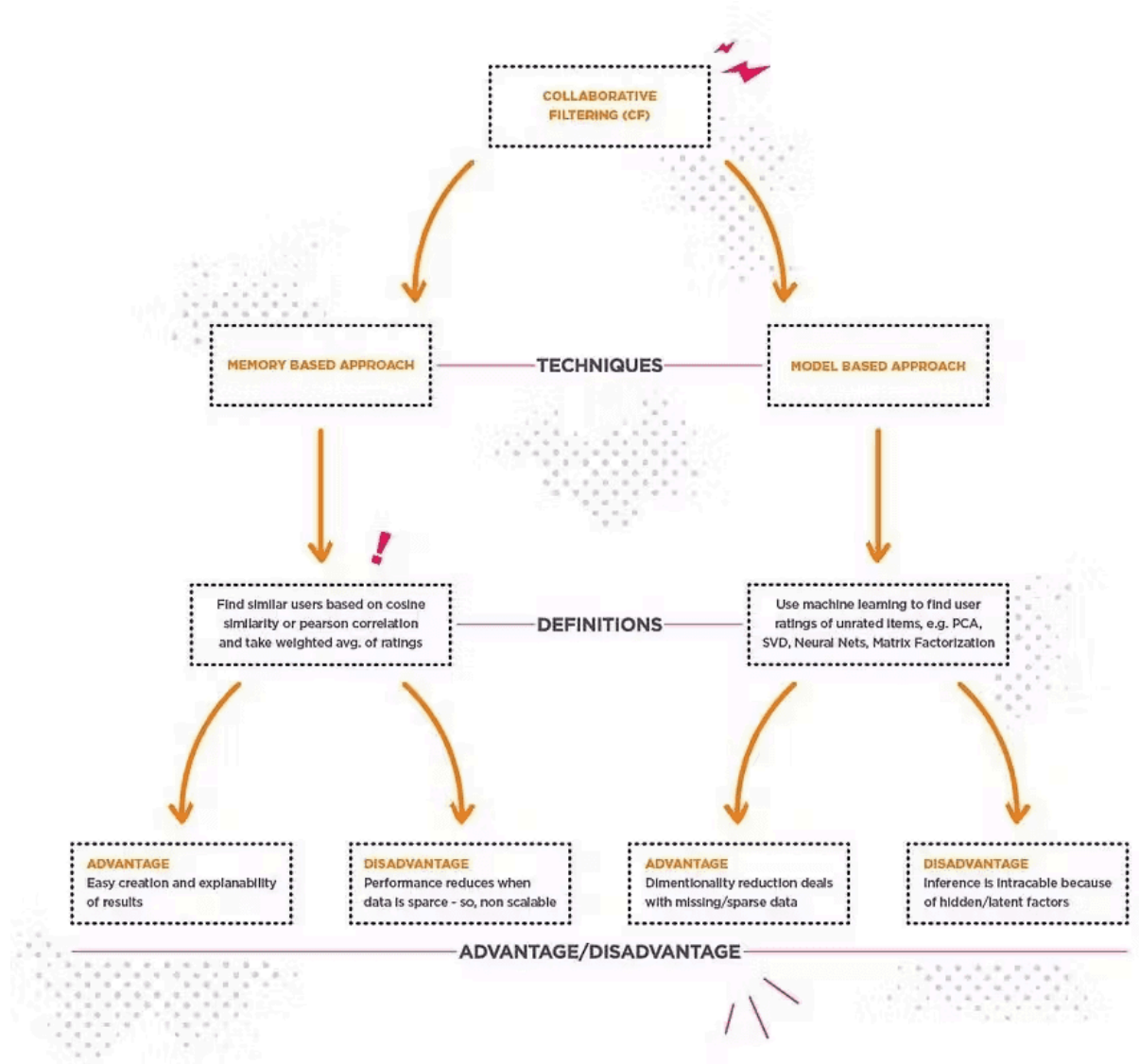


Image 10 – Memory-Based vs. Model-Based CF

User-Based Collaborative Filtering

This method recommends items to a user based on the preferences of similar users in the database. It involves creating a matrix of items

rated/liked/clicked by each user, computing similarity scores between users, and then suggesting items that the target user hasn't encountered yet but similar users have liked.

Example – If User A likes 'Batman Begins' and 'Justice League,' and User B likes 'Batman Begins,' 'Justice League,' and 'Avengers,' User A might also enjoy 'Avengers.'

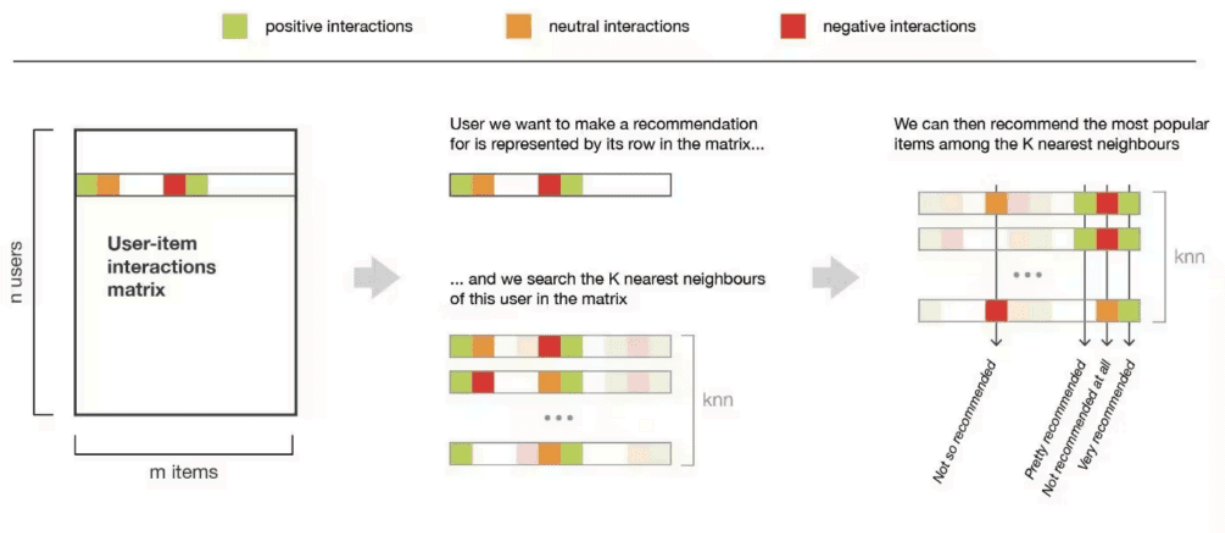


Image 11 – User-Based Collaborative Filtering

Item-based Collaborative Filtering

Instead of finding similar users, this method finds similar items to recommend based on the user's past preferences. It identifies pairs of items rated/liked by the same users, measures their similarity, and then suggests similar items based on these similarity scores.

Example – If User A liked 'Movie 1' and 'Movie 2', and 'Movie 2' is similar to 'Movie 3' based on ratings by other users, 'Movie 3' is recommended to User A.

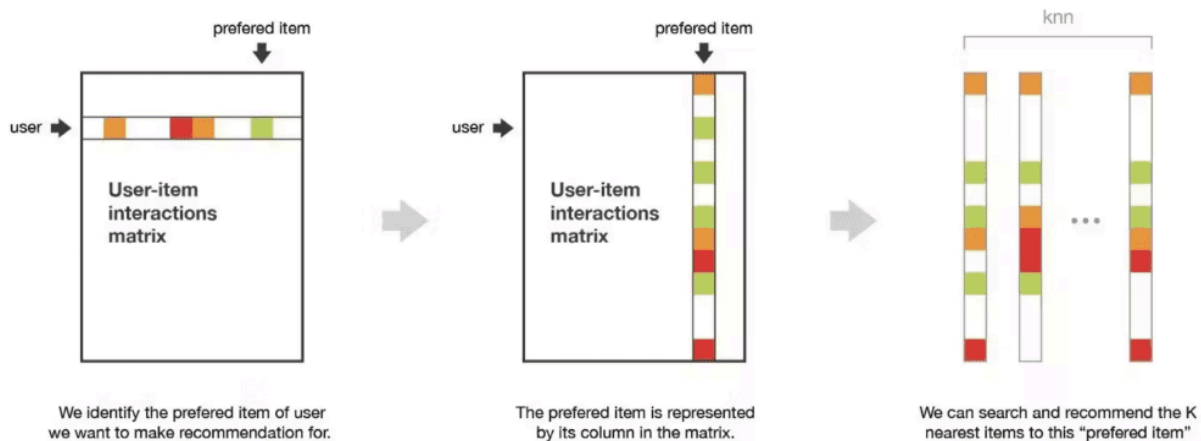


Image 12 – Item-Based Collaborative Filtering

Recommendation Methods

Matrix Factorization – In this method, the sparse user-item interaction matrix is divided into two smaller, dense matrices representing the latent features of users and items. In the context of movie recommendations, latent features could represent different genres or user preferences for certain types of movies.

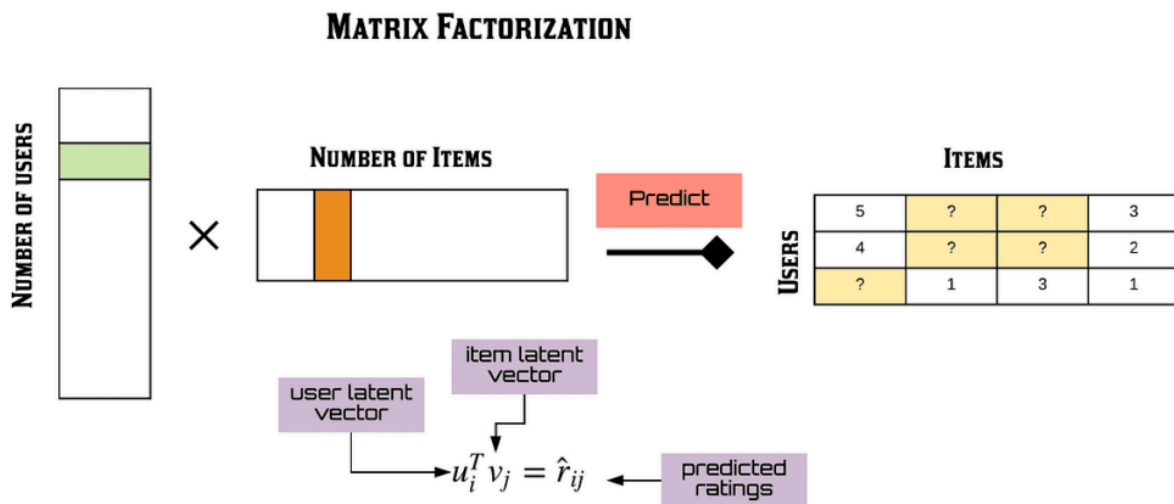


Image 13 – Matrix Factorization

K-Nearest Neighbors (KNN) – To make a recommendation, KNN identifies the “neighbors” of a user or item, which are most similar based on certain features or past behavior. For example, KNN might compare users based on their movie ratings in a movie recommendation system. If a user has given similar ratings to a set of movies as another user, KNN considers them

neighbors. The system then recommends movies that the neighbor liked, but the user hasn't seen yet.

K Nearest Neighbors

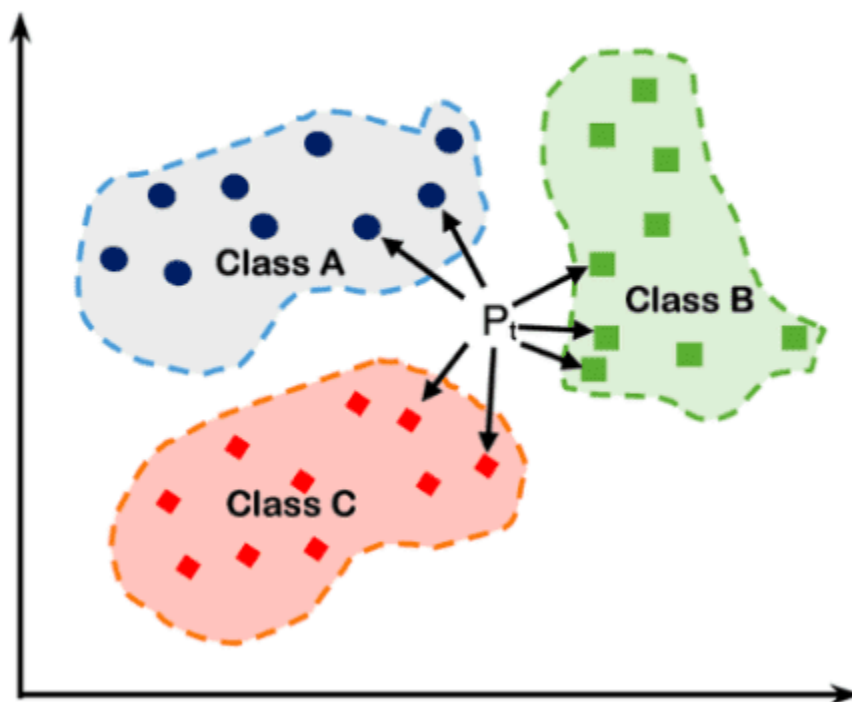


Image 14 – K

Nearest Neighbors

The process involves several steps. First, the system computes the similarity between users or items using metrics like Euclidean distance or cosine similarity. Then, it selects the top 'k' most similar neighbors. A user-based KNN finds users with similar tastes; an item-based KNN identifies items that

have been similarly rated. Finally, the system aggregates the preferences of these neighbors to generate a recommendation list.

Early recommendation models often use collaborative filtering, [K-Nearest-Neighbor \(KNN\)](#) models predict a user's preferences by comparing them to similar users or items. User-user-based collaborative filtering uses a target user's neighbors' preferences, while item-item-based filtering uses a user's preferences on similar items. Combining both methods improves accuracy in handling large datasets, and algorithms like SVD reduce data dimensions. [Matrix Factorization \(MF\)](#) considers implicit feedback and temporal information, outperforming KNN.

Context Filtering

Session-based and sequence-based recommender systems predict the next user action by analyzing past interactions. Session-based systems focus on the current session's actions, while sequence-based systems use the order of all past interactions. These systems face challenges like varying session lengths, action types, and user anonymity. Conventional methods include K-nearest neighbors and Markov chains, while advanced approaches use deep learning models like RNNs and [attention mechanisms](#). Netflix, for example,

uses contextual sequence prediction by considering user actions and context to suggest what to watch next.

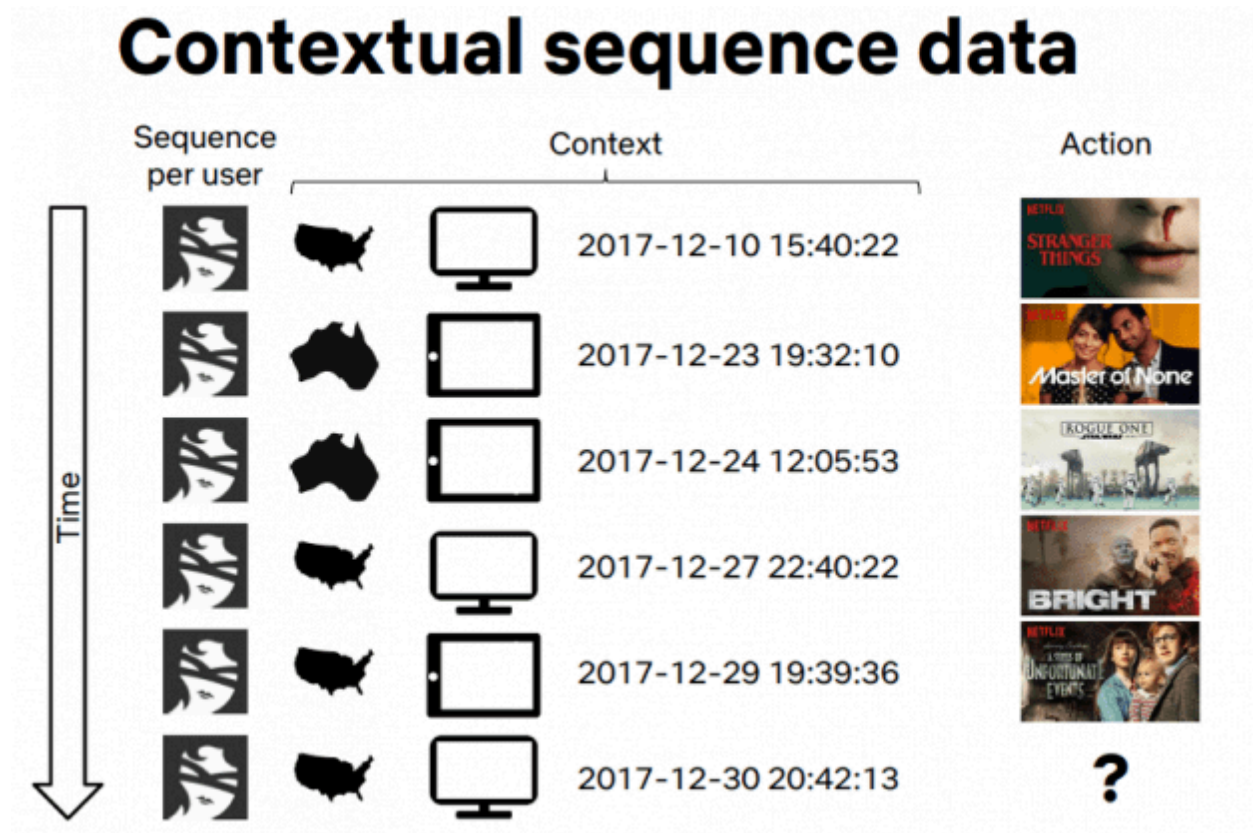


Image 15 – Context Filtering

This figure shows how user interactions with a service like Netflix can be tracked over time with additional context to improve recommendations. Each row represents a user's activity at a specific time, including their location, the device they used, and the exact date and time of the interaction. For example, a user in the United States watched "Stranger Things" on their computer on

December 10, 2017. The system captures more information about each user's behavior by recording this context—location, device, and timestamp.

With this detailed data, recommendation systems can make better predictions about what users might want to watch next. For instance, if the system sees that a user often watches action movies on their computer in the evening, it can suggest similar movies around that time. The question mark in the last row shows an unknown action the system aims to predict based on past interactions and context. This approach helps create personalized recommendations that match the user's habits and preferences, enhancing their viewing experience.

Summarization of Recommendation System Types

Content-based recommendation systems(or recommender systems) suggest items by matching item attributes with user preferences, relying on techniques like TF-IDF for profiling. Collaborative filtering leverages users' collective behaviors to make predictions, employing methods such as user-based and item-based filtering and advanced techniques like matrix factorization.

Context filtering predicts user actions based on session and sequence data, utilizing models like RNNs and attention mechanisms to provide relevant

recommendations. Now, let's move to the next section to explore the main components behind these recommendation systems' workflows.

Recommendation System – Traditional ML Approaches

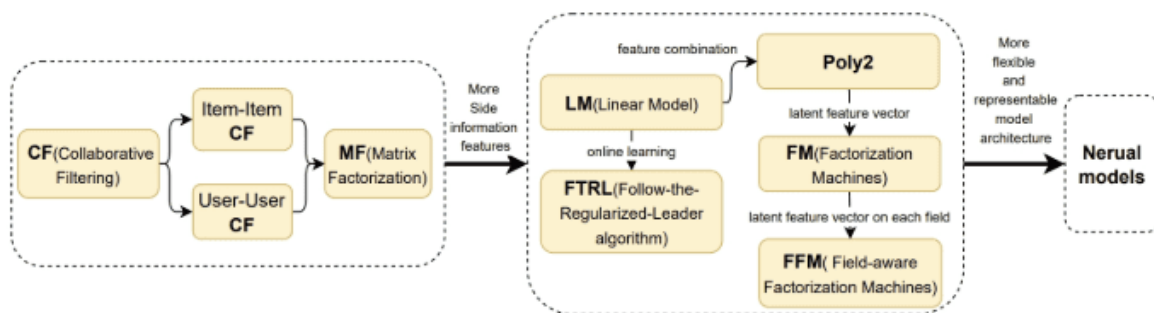


Image 16 – ML Approach Evaluation in RecSys

Many advancements have occurred throughout the decades for recommendation systems. Here, we will explore three main and widely used ML techniques.

Matrix Factorization

As discussed, matrix factorization decomposes the matrix into two lower-dimensional matrices representing users and items. These lower-dimensional matrices capture latent features that describe the preferences of users and the characteristics of items. After that, latent features are used to predict user recommendations. Now, two different types of algorithms have been used based on the feedback we received.

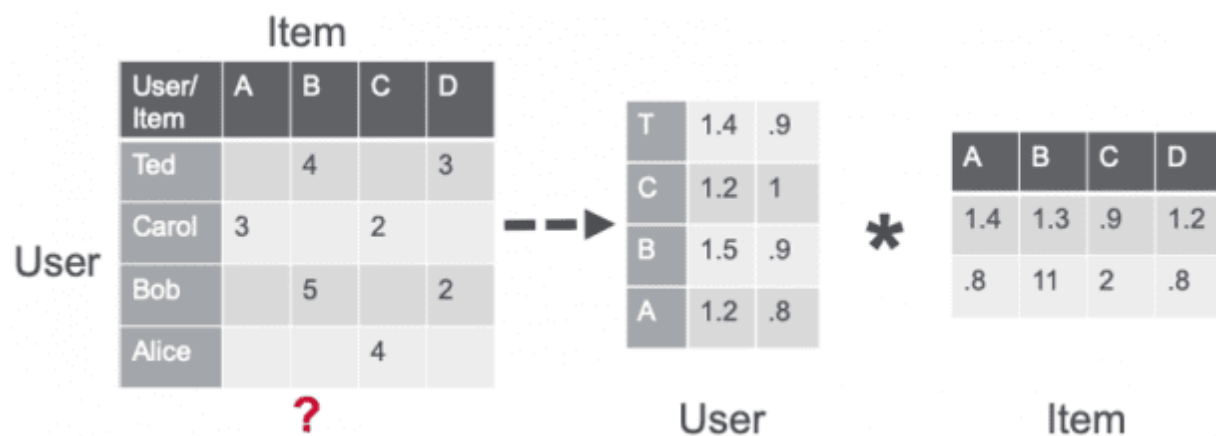


Image 17 – Matrix Factorization Process

For Implicit Feedback

The SVD (singular value decomposition) algorithm is used for implicit feedback, which involves measuring user confidence in their preferences. Suppose t_{ui} represents how much of a movie (i) user (u) has watched. For example, $t_{ui} = 0$ means the user never watched the movie, $t_{ui} = 0.3$ means they

watched 30% of it, and $t_{ui} = 2$ means they watched it twice. This implies that a user is likely more interested in a movie they've watched two times than in one they haven't watched at all.

We define the confidence matrix c_{ui} and the rating matrix r_{ui} as follows: The confidence matrix is $c_{ui} = 1 + \alpha t_{ui}$, where α is a constant. The rating matrix r_{ui} is defined as $r_{ui} = 1$ if $t_{ui} > 0$ (indicating the user has watched the movie) and $r_{ui} = 0$ if $t_{ui} = 0$ (indicating the user has not watched the movie).

$$c_{ui} = 1 + \alpha t_{ui} \quad \text{and} \quad r_{ui} = \begin{cases} 1 & \text{if } t_{ui} > 0 \\ 0 & \text{if } t_{ui} = 0 \end{cases}$$

Image 18 –

confidence matrix and rating matrix for implicit feedback

Then, using both matrices, t computes the loss of overall user-item pairs (u, i) using a weighted squared error between the observed rating r_{ui} and the predicted rating $\hat{r}_{ui}$, weighted by the confidence c_{ui} . Regularization terms for user and item feature vectors (p_u and q_i) are added to prevent overfitting and scaled by a parameter λ . If a user never interacted with an item, r_{ui} is set to 0.

$$L = \sum_{\text{all } u, i} \left[c_{ui} (r_{ui} - \hat{r}_{ui})^2 + \lambda \left(\|p_u\|_2^2 + \|q_i\|_2^2 \right) \right]$$

Image 19 – Loss function for implicit feedback

For Explicit feedback

Explicit feedback uses direct user ratings for items, such as movie ratings. In matrix factorization for explicit feedback, a linear model represents user-item interactions, and the algorithm, called probabilistic matrix factorization, learns latent vectors for users and items by minimizing a regularized mean squared error (MSE) loss over known ratings.

$$\hat{r}_{ui} = q_i^T p_u$$

$$L = \sum_{(u,i) \in K} \left[(r_{ui} - \hat{r}_{ui})^2 + \lambda \left(\|p_u\|_2^2 + \|q_i\|_2^2 \right) \right]$$

Image 20

– Probabilistic Matrix Factorization *and Loss function* for explicit feedback

Two common optimization methods are used: stochastic gradient descent (SGD) and alternating least squares (ALS). While SGD is easy to implement, it

may struggle with non-convex loss functions. ALS, however, can transform the problem into a series of convex linear regression problems, which are easier to solve and can be significantly parallelized for speed.

Hybrid Approach

The SVD++ algorithm can handle both explicit and implicit feedback simultaneously, which is useful because users often interact with many items but rate only a few. This algorithm modifies the basic SVD model by including a weighted sum of latent factors from items a user has interacted with. This helps provide more accurate recommendations by leveraging all available user interactions, not just the rated items.

$$\hat{r}_{ui} = \mu + b_u + b_i + q_i^T \left(p_u + |N(u)|^{-0.5} \sum_{j \in N(u)} z_j \right)$$

Image 21 – SVD++ for mixed (explicit + implicit) feedback

Matrix factorization methods, especially those using techniques like ALS and SVD, have proven highly effective in collaborative filtering systems. They are

more advanced than traditional methods like K-nearest neighbors (KNN) due to their ability to handle large datasets.

Logistic Regression

Logistic regression is a commonly used linear model in recommendation systems, especially for predicting click-through rates (CTR). This model predicts the probability of an event occurring, with values between 0 and 1. It can use side information such as user demographics, past behaviors, item attributes, and contextual information, helping to address the cold start problem. The logistic regression model is trained by minimizing a loss function with two parts: a regularization term and a logistic loss term. The optimization problem can be expressed as:

$$\min_{\mathbf{w}} \frac{\lambda}{2} \|\mathbf{w}\|_2^2 + \sum_{i=1}^n \log(1 + \exp(-y_i \phi(\mathbf{w}, \mathbf{x}_i))),$$

Image 22 – Logistic Regression

Here, λ is the regularization parameter, $\mathbf{w}$ is the weight vector, y_i is the label, and $\mathbf{x}_i$ is the feature vector.

Traditional logistic regression assigns a weight to each feature but does not consider how features interact with each other. Additional techniques are used to capture these interactions. One method is the degree-2 polynomial (Poly2) model, which learns a weight for every possible feature pair. Another

approach combines decision trees with logistic regression, where decision trees transform input features, and their outputs serve as inputs to the logistic model.

$$\phi_{\text{FM}}(\mathbf{w}, \mathbf{x}) = \sum_{j_1=1}^n \sum_{j_2=j_1+1}^n (\mathbf{w}_{j_1} \cdot \mathbf{w}_{j_2}) x_{j_1} x_{j_2},$$

Image 23 – Factorization Machines

Field-aware Factorization Machines (FFMs) extend the basic FM model by grouping features into different fields. Each feature has a different latent vector for each field, allowing the model to capture more nuanced interactions between features from different fields. In FFMs, when deciding the weight for a feature pair, the latent vector of one feature in the pair is used in the context of the field of the other feature and vice versa.

$$\phi_{\text{FFM}}(\mathbf{w}, \mathbf{x}) = \sum_{j_1=1}^n \sum_{j_2=j_1+1}^n (\mathbf{w}_{j_1, f_2} \cdot \mathbf{w}_{j_2, f_1}) x_{j_1} x_{j_2},$$

Image 24 – Field-aware Factorization

Machines

These algorithms mostly use feature vectors to provide predictions as recommendations, and a ranking method like similarity search ranks those features according to user input. However, these models face challenges when trying to model interactions involving more than two features. This limitation has led researchers to explore neural network-based recommendation systems, which offer more flexibility for advanced feature

interactions and can more effectively handle higher-order feature combinations.

Factorization Machines (FMs)

Factorization Machines (FMs) are a type of generalized low-rank model used to approximate tabular datasets by representing them with lower-dimensional latent factors. Unlike traditional matrix factorization, FMs can handle more complex feature interactions, making them useful for tasks like recommendation systems. FMs convert the weight of each feature pair into the inner product of two k-dimensional latent vectors corresponding to each feature in the pair.

$$\phi_{\text{FM}}(\mathbf{w}, \mathbf{x}) = \sum_{j_1=1}^n \sum_{j_2=j_1+1}^n (\mathbf{w}_{j_1} \cdot \mathbf{w}_{j_2}) x_{j_1} x_{j_2},$$

Image 23 – Factorization Machines

Field-aware Factorization Machines (FFMs) extend the basic FM model by grouping features into different fields. Each feature has a different latent vector for each field, allowing the model to capture more nuanced interactions between features from different fields. In FFMs, when deciding the weight for a feature pair, the latent vector of one feature in the pair is used in the context of the field of the other feature and vice versa.

$$\phi_{\text{FFM}}(\mathbf{w}, \mathbf{x}) = \sum_{j_1=1}^n \sum_{j_2=j_1+1}^n (\mathbf{w}_{j_1, f_2} \cdot \mathbf{w}_{j_2, f_1}) x_{j_1} x_{j_2},$$

Image 24 – Field-aware Factorization

Machines

These algorithms mostly use feature vectors to provide predictions as recommendations, and a ranking method like similarity search ranks those features according to user input. However, these models face challenges when trying to model interactions involving more than two features. This limitation has led researchers to explore neural network-based recommendation systems, which offer more flexibility for advanced feature interactions and can more effectively handle higher-order feature combinations.

DNN-Based Recommendation Systems

Deep neural networks have gradually improved recommendation systems by understanding complex relationships between users and items. We will explore how embeddings turn data into useful patterns, different network types like MLPs and CNNs, and popular models such as Neural Collaborative Filtering, Variational Autoencoders, Google's Wide and Deep model, and Meta's Deep Learning Recommender Model. Before that, we will see what the main components work under the hood to make such complicated and accurate architectures.

Key Components

Embeddings

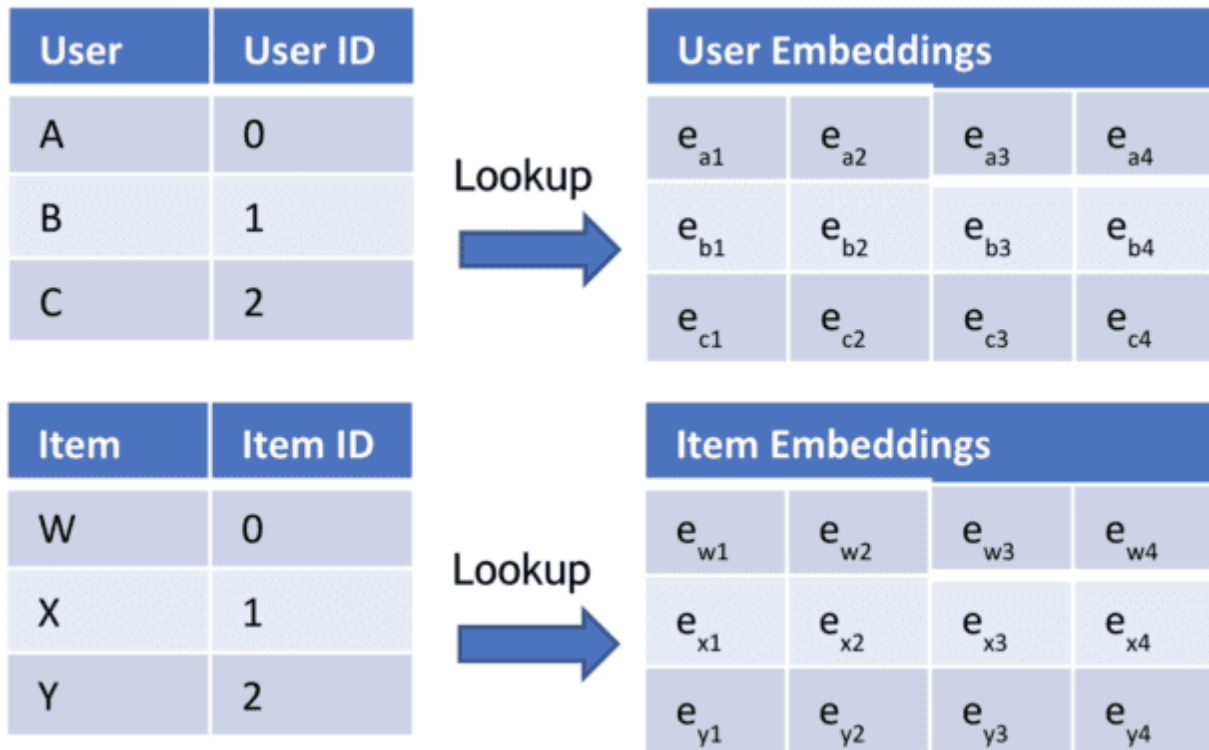


Image 29 – Embeddings

Embeddings are a core component of DL recommender systems, transforming categorical data into dense vector representations. Using Embeddings, the model captures similarities between entities, such as users and items, in a high-dimensional space. For example, users with similar preferences will have similar embedding vectors. These embeddings are learned during training and can significantly enhance the model's ability to generalize from sparse data.

Core Architecture

DL recommender systems utilize various network architectures, including feedforward neural networks, multilayer perceptrons (MLPs), convolutional

neural networks (CNNs), and recurrent neural networks (RNNs). Feedforward networks pass information forward through layers, while MLPs add depth and non-linearity with hidden layers. CNNs are used for processing image data, and RNNs handle sequential data, such as user interaction histories.

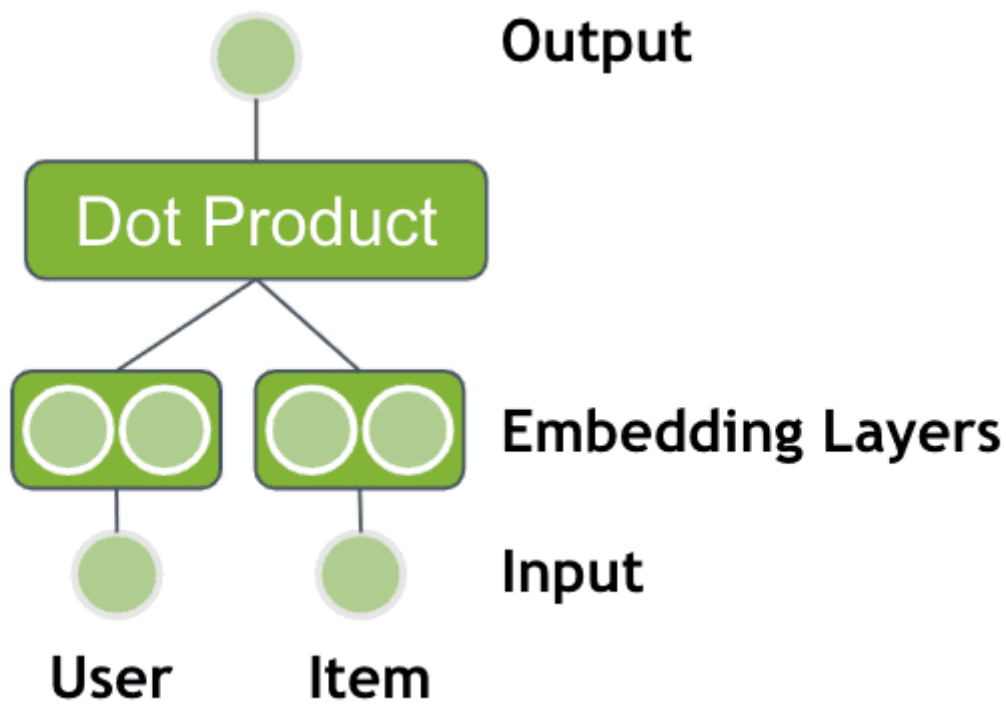


Image 30 – Core Architecture

The core architecture of these neural networks calculates the dot-product between the user embedding and the item embedding to get a final score, the likelihood that a user interacts with an item. As a last step, you may apply the sigmoid activation function to transform the output to a probability between 0 and 1.

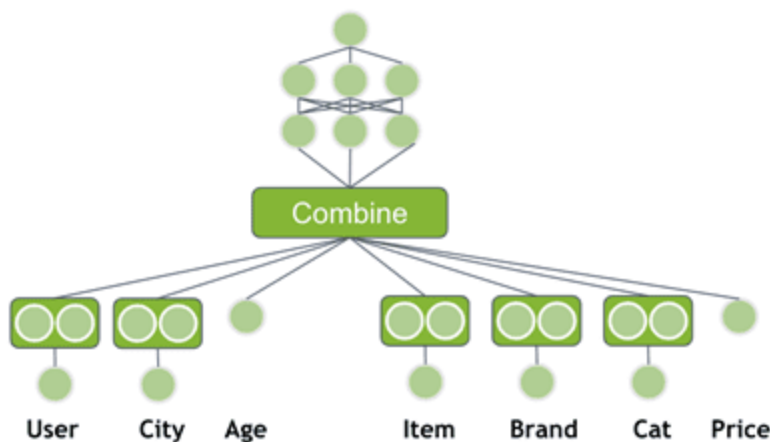


Image 31 – Core Architecture

with Multiple Inputs

It's important to consider additional user information such as gender, age, city, time since last visit, and credit card used for payment, along with item details like brand, price, categories, and quantity sold in the last seven days. This additional information can enhance the model's ability to generalize. Modify the neural network to incorporate these extra features as inputs.

Popular Architectures

Neural Collaborative Filtering (NCF)

NCF combines matrix factorization with MLPs to model user-item interactions. It treats the problem from a non-linearity perspective, where user and item embeddings are learned through interaction data. The model then multiplies these embeddings and feeds them into an MLP network. The matrix factorization and MLP outputs are combined to predict interaction probabilities. This approach allows for capturing complex, non-linear relationships in the data, enhancing recommendation accuracy.

Key Takeaways

Understanding Recommendation System

Recommendation systems are algorithms that suggest relevant items to users based on their preferences and behaviors. They are crucial for platforms like Spotify, Amazon, and Netflix to help users find songs, products, or shows they might like. These systems analyze user interactions, such as clicks and purchases, to predict what users will enjoy. By understanding user and item profiles, recommendation systems make personalized suggestions, enhancing user satisfaction and engagement.

Types of Recommendation System

There are three main types of recommendation systems: content-based, collaborative filtering, and context filtering. Content-based systems match item attributes with user preferences, while collaborative filtering uses user interactions to find similarities among users or items. Context filtering considers the sequence of user actions to predict future behavior. Each type has unique methods and applications, providing various solutions for different recommendation challenges.

Traditional Machine Learning Techniques

Traditional machine learning techniques like matrix factorization, logistic regression, and factorization machines play a significant role in recommendation systems. Matrix factorization decomposes user-item interaction matrices into lower-dimensional representations, capturing latent features. Logistic regression predicts click-through rates by analyzing features and interactions. Factorization machines extend matrix factorization by modeling feature interactions, enhancing prediction accuracy. These techniques form the foundation of many recommendation systems.

Advancements with Deep Learning

Deep learning models have revolutionized recommendation systems by handling large datasets and complex interactions. Techniques like Neural Collaborative Filtering (NCF), Variational Autoencoders (VAE), and the Wide and Deep model combine user and item embeddings with neural networks to capture non-linear relationships. These models improve accuracy and personalization by learning intricate patterns from user behavior. Additionally, session-based recommendations using RNNs and transformers provide real-time suggestions, enhancing user experience on platforms like Netflix and Google.

Emerging Trends with Large Language Models

Large Language Models (LLMs) like ChatGPT and BERT enhance recommendation systems by understanding and generating natural language. These models can handle complex decision-making processes and improve user experience by making recommendations more accurate and explainable.

Conclusion

A recommendation system(or recommender system) is key to enhancing user experience on modern platforms. By understanding user preferences and behaviors, these systems offer personalized suggestions that engage users. Recommendation systems have evolved from traditional methods like matrix factorization to advanced deep learning models to provide accurate and relevant recommendations. They are essential for navigating the vast amount of content available today, ensuring users find what they need quickly and easily. So, the next time your parents get their favorite song or movie suggested as “you might like this,” you can tell them the secret behind it.

